myshell

```c
#include <stdio.h>
#include <string.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <sys/types.h>
#include <fcntl.h>


#define MAXLINE 4096
#define MAXPIPE 16
#define MAXARG 8


struct {
     char *argv[MAXARG];
     char *in, *out;
} cmd[MAXPIPE+1];


int parse(char *buf, int cmdnum)
{
     int n = 0;
     char *p = buf;


     cmd[cmdnum].in = cmd[cmdnum].out = NULL;
     while (*p != '\0')
     {
          if (*p == ' ')
          {
               *p++ = '\0';
               continue;
          }
          if (*p == '<')
          {
               *p = '\0';
               while (*(++p) == ' ');
               cmd[cmdnum].in = p;
               if (*p++ == '\0')
```

```
                if ( p++ == '\0' )
                    return -1;
                continue;
            }
            if (*p == '>')
            {
                *p = '\0';
                while (*(++p) == ' ');
                cmd[cmdnum].out = p;
                if (*p++ == '\0')
                    return -1;
                continue;
            }
            if (*p != ' ' && ((p == buf) || *(p-1) == '\0'))
            {
                if (n < MAXARG - 1) {
                    cmd[cmdnum].argv[n++] = p++;
                    continue;
                } else
                    return -1;
            }
            p++;
        }
        if (n == 0)
            return -1;
        cmd[cmdnum].argv[n] = NULL;
        return 0;
}


int main(void)
{
        char buf[MAXLINE];
        pid_t pid;
        int fd, i, j, pfd[MAXPIPE][2], pipe_num, cmd_num;
        char* curcmd, *nextcmd;


        while (1) {
         printf("mysh%% ");
         if (!fgets(buf, MAXLINE, stdin))
              exit(0);

         if (buf[strlen(buf)-1]=='\n')
```

```
84            buf[strlen(buf)-1]='\0';
85       cmd_num = 0;
86       nextcmd = buf;
87       while (curcmd = strsep(&nextcmd, "|")) {
88            if (parse(curcmd, cmd_num++)<0) {
89             cmd_num--;
90             break;
91            }
92            if (cmd_num == MAXPIPE + 1)
93             break;
94       }
95       if (!cmd_num)
96            continue;
97           pipe_num = cmd_num - 1;
98
99
100      for (i=0; i<pipe_num; i++)
101           if (pipe(pfd[i])) {
102            perror("pipe");
103            exit(1);
104           }
105      for (i=0; i<cmd_num; i++)
106           if ((pid=fork())==0)
107            break;
108
109
110      if (pid==0) {
111           if (pipe_num) {
112            if (i==0) {
113             dup2(pfd[0][1], STDOUT_FILENO);
114             close(pfd[0][0]);
115             for (j=1; j<pipe_num; j++) {
116                  close(pfd[j][0]);
117                  close(pfd[j][1]);
118             }
119            } else if (i==pipe_num) {
120             dup2(pfd[i-1][0], STDIN_FILENO);
121             close(pfd[i-1][1]);
122             for (j=0; j<pipe_num-1; j++) {
123                  close(pfd[j][0]);
124                  close(pfd[j][1]);
125             }
126            } else {
127             dup2(pfd[i-1][0], STDIN_FILENO);
```

```
128             close(pfd[i-1][1]);
129             dup2(pfd[i][1], STDOUT_FILENO);
130             close(pfd[i][0]);
131             for (j=0; j<pipe_num; j++)
132                  if (j!=i || j!=i-1) {
133                   close(pfd[j][0]);
134                   close(pfd[j][1]);
135                  }
136            }
137           }
138           if (cmd[i].in) {
139            fd = open(cmd[i].in, O_RDONLY);
140            if (fd != -1)
141             dup2(fd, STDIN_FILENO);
142           }
143           if (cmd[i].out) {
144            fd = open(cmd[i].out, O_WRONLY|O_CREAT|O_TRUNC, 0644);
145            if (fd != -1)
146             dup2(fd, STDOUT_FILENO);
147           }
148           execvp(cmd[i].argv[0], cmd[i].argv);
149           fprintf(stderr, "executing %s error.\n", cmd[i].argv[0]);
150           exit(127);
151       }
152
153
154
155
156       for (i=0; i<pipe_num; i++) {
157            close(pfd[i][0]);
158            close(pfd[i][1]);
159       }
160       for (i=0; i<cmd_num; i++)
161            wait(NULL);
162      }
163 }
```